

# Assignment: “Fair Split”

**Due in 1 day. Stack: your choice. Deliverable: a deployed API + a minimal frontend.**

---

## What you’re building

A tool that takes a photo of a restaurant bill and a plain-English description of who had what, and returns a fair, fully-reconciled, per-person breakdown. That includes tax, service charge, any discount, and a final “who owes whom” settle-up.

Example description: *“Aman skipped drinks. Priya and I shared the pasta. The cheesecake was Karan’s. Everything else was common to all four of us. Priya paid the bill.”*

## Input

- **Receipt image** (we provide sample images; see below)
- **Free-text “who had what” description**, which also names who paid the bill

## Request contract (your endpoint must accept this exactly)

Your deployed endpoint must accept a POST with a JSON body (Content-Type: application/json):

```
{
  "receipt_base64": "<base64-encoded image bytes, no data-URI prefix>",
  "description": "<the plain-English string>"
}
```

It must respond with application/json in exactly the output shape below.

## Output — your API must return exactly this shape

```
{
  "per_person": [
    {
      "name": "Aman",
      "items": ["Pasta (½)"],
      "subtotal": 220,
      "tax_share": 28,
      "service_share": 22,
      "discount_share": -20,
      "total": 250
    },
    {
      "name": "Priya",
      "items": ["Cheesecake"],
      "subtotal": 100,
      "tax_share": 12,
      "service_share": 10,
      "discount_share": 0,
      "total": 122
    },
    {
      "name": "Karan",
      "items": ["Beverages"],
      "subtotal": 50,
      "tax_share": 6,
      "service_share": 5,
      "discount_share": 0,
      "total": 61
    },
    {
      "name": "Sara",
      "items": ["Beverages"],
      "subtotal": 50,
      "tax_share": 6,
      "service_share": 5,
      "discount_share": 0,
      "total": 61
    }
  ],
  "grand_total": 1000,
  "reconciliation": {
    "sum_of_person_totals": 1000,
    "matches_bill": true
  },
  "paid_by": "Priya",
  "settle_up": [
    {
      "from": "Aman",
      "to": "Priya",
      "amount": 250
    },
    {
      "from": "Karan",
      "to": "Priya",
      "amount": 61
    },
    {
      "from": "Sara",
      "to": "Priya",
      "amount": 61
    }
  ],
  "assumptions": ["'rest of us' interpreted as Aman, Priya, Karan, Sara"],
}
```

```
"flags": ["Extracted line items sum to ₹980 but printed total is ₹1000 – ₹20 unexplained"]
}
```

The reconciliation, assumptions, and flags fields are mandatory. They force your system to check its own arithmetic and to surface anything that doesn't add up, instead of silently guessing.

The payer is named in the description. If no payer is stated, flag it rather than assume one.

## Fairness rules (use these exactly, for fairness across candidates)

1. Each person pays for the items they consumed.
  2. Shared items split **equally among the people who shared that specific item**.
  3. Tax + service charge allocated **proportional to each person's pre-tax food subtotal**.
  4. A bill-level discount allocated **proportional to subtotal**.
  5. Round to the rupee; state in assumptions who absorbs the leftover paise.
- 

## These 4 receipts are only samples

You're given **4 sample receipts** to develop against. Real bills are messier than these, and we will evaluate your tool on receipts you have never seen, including some awkward ones.

Part of this assignment is anticipating what we'll throw at it. Source your own receipts (real photos, or ones you construct) and stress-test your tool against cases the samples don't cover. Some dimensions worth probing (this list isn't exhaustive, and we want you to find more): - bills with no service charge - bills where the printed total doesn't add up - a description that mentions an item not on the bill - ambiguous wording like "the rest of us" - shared items across only a subset of people - quantities that don't divide evenly - tips or charges the fairness rules don't mention - multiple people owing one payer

## Required: an Edge Cases section

In your submission, include a section that lists every edge case you considered, and for each one: - what the input looks like, - how your tool handles it (or that you chose not to handle it, and why), - whether you verified the behavior.

The range and honesty of this list matters more to us than handling every case. "I detected this case and chose to flag it rather than guess" is a strong answer. A tool that silently returns a confident wrong number is the thing we're testing against.

---

## Deliverables

1. **Deployed API + minimal frontend.** Upload an image, paste a description, see the result table. Any framework, any host.
2. **Prompt log.** Your prompt iterations, one line each on what changed and why. Answer one question explicitly: did you let the model do the arithmetic, or extract structured data and compute the totals in code? Why?
3. **Edge-case doc**, as described above.
4. **“Where the AI was wrong” note.** 3 concrete examples from your own testing where the model’s first answer was wrong (a misread price, a botched total, a hallucinated item) and how you caught and fixed it.

## Ground rules

- Use any model, any framework, any deploy target.
  - Don’t build auth, history, or persistence. One bill in, one split out.
  - If an input is ambiguous or doesn’t reconcile, flag it rather than fabricate an answer.
- 

## Sample receipts (R1–R4)

Convention on every bill: **Service Charge = 5% of subtotal** (unless the bill says otherwise), **GST = 5% (CGST 2.5% + SGST 2.5%) on (subtotal + service – discount)**, rounded to a whole-rupee grand total.

### R1 — Brew & Bite Café, Koramangala, Bengaluru · 12 Mar 2026 · Bill #0142

| Item                     | Qty | Amount |
|--------------------------|-----|--------|
| Cappuccino               | 1   | 180    |
| Grilled Chicken Sandwich | 1   | 260    |
| Penne Arrabiata          | 1   | 320    |
| Fresh Lime Soda          | 1   | 120    |
| Brownie                  | 1   | 160    |

Subtotal 1040 · Service 5% 52 · GST 54.60 · Round-off +0.40 · **Grand Total ₹1147**

**Description:** “Three of us — Ravi, Neha, Sameer. Ravi had the cappuccino and the sandwich. Neha had the pasta and the lime soda. Sameer had the brownie. Sameer paid.”

## R2 — Tamarind Kitchen, HSR Layout, Bengaluru · 14 Mar 2026 · Bill #2207

| Item                 | Qty  | Amount |
|----------------------|------|--------|
| Paneer Butter Masala | 1    | 320    |
| Dal Makhani          | 1    | 260    |
| Butter Naan          | 4    | 240    |
| Jeera Rice           | 1    | 180    |
| Gulab Jamun          | 2 pc | 120    |
| Masala Papad         | 2    | 100    |

Subtotal 1220 · Service 5% 61 · GST 64.05 · Round-off -0.05 · **Grand Total ₹1345**

**Description:** *"Four of us: Aman, Priya, Karan, Sara. The Gulab Jamun was shared just by Priya and Karan. Everything else was common to all four. Priya paid."*

## R3 — The Daily Grind, Powai, Mumbai · 15 Mar 2026 · Bill #1188

| Item             | Qty | Amount |
|------------------|-----|--------|
| Margherita Pizza | 1   | 380    |
| Arrabiata Pasta  | 1   | 340    |
| Garlic Bread     | 1   | 160    |
| Craft Beer       | 2   | 500    |
| Virgin Mojito    | 1   | 180    |

Subtotal 1560 · Service 5% 78 · GST 81.90 · Round-off +0.10 · **Grand Total ₹1720**

**Description:** *"Ishaan, Meera, Rohit. Pizza, pasta and garlic bread shared equally by all three. The two beers were Ishaan and Rohit only. The mojito was Meera's. Rohit paid."*

## R4 — Spice Route, Jubilee Hills, Hyderabad · 16 Mar 2026 · Bill #5521

| Item              | Qty | Amount |
|-------------------|-----|--------|
| Chicken Biryani   | 2   | 560    |
| Veg Biryani       | 1   | 240    |
| Mutton Rogan Josh | 1   | 420    |
| Raita             | 2   | 120    |
| Soft Drinks       | 3   | 180    |

Subtotal 1520 · Discount "WELCOME15" -15% -228 · Service 5% 76 · GST 68.40 ·

Round-off -0.40 · **Grand Total ₹1436** **Description:** *"Dev and Nikhil each had a chicken biryani. Anjali had the veg biryani. Farah had the rogan josh. The raita and soft drinks were common to all four. We used a 15% off coupon. Anjali paid."*